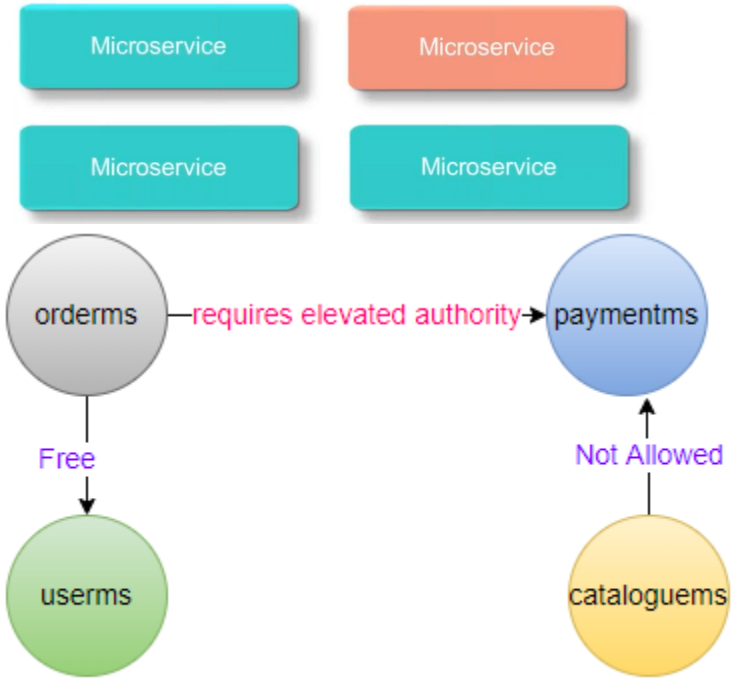
Security:

Securing Microservices: Our data is our responsibility

Segregate Sensitive Information

- Sensitive information needs to be segregated.
- The number of microservices that will process this data must be reduced as much as possible.
- Dedicated microservices can be created for processing, storing customer personal identifiable information and securing the personal customer data as no other component will have access to it.
- Microservice reduces the risk of data mishap through **Isolation**.



Introduce Security Levels

- Each microservices has –
  - Different bounded contexts,
  - Varying amounts of capabilities &
  - Different purposes, being implemented with heterogeneous tools and languages by different teams.

Microservices could be handling highly sensitive data like security tokens and credit cards, or public information which is easily available or be somewhere in the middle of most secure and public data.

Best Practices

Provide Least Privilege

- Use the principal of Least Privilege - provide the minimal access that is required by one service to complete the job.

Discover and monitor inter-service communications

- Observing inter-service communication is a way to establish the baseline for policies by understanding application behavior. Service discovery is the process which enables automatic identification of new services to others entities in the application.

Best Practices

Segment and isolate applications and services

Microservice segmentation is a policy-governed walling of applications by applying granular security enforcement

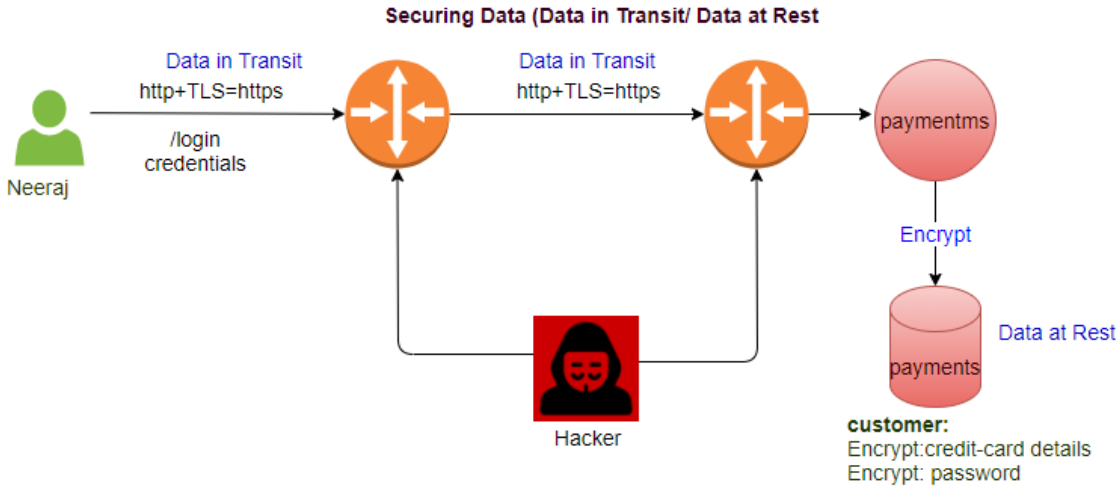
Encrypt data in transit and at rest

Encrypt traffic between services to ensure compliance requirements are met
Improve security, especially when the traffic crosses public networks

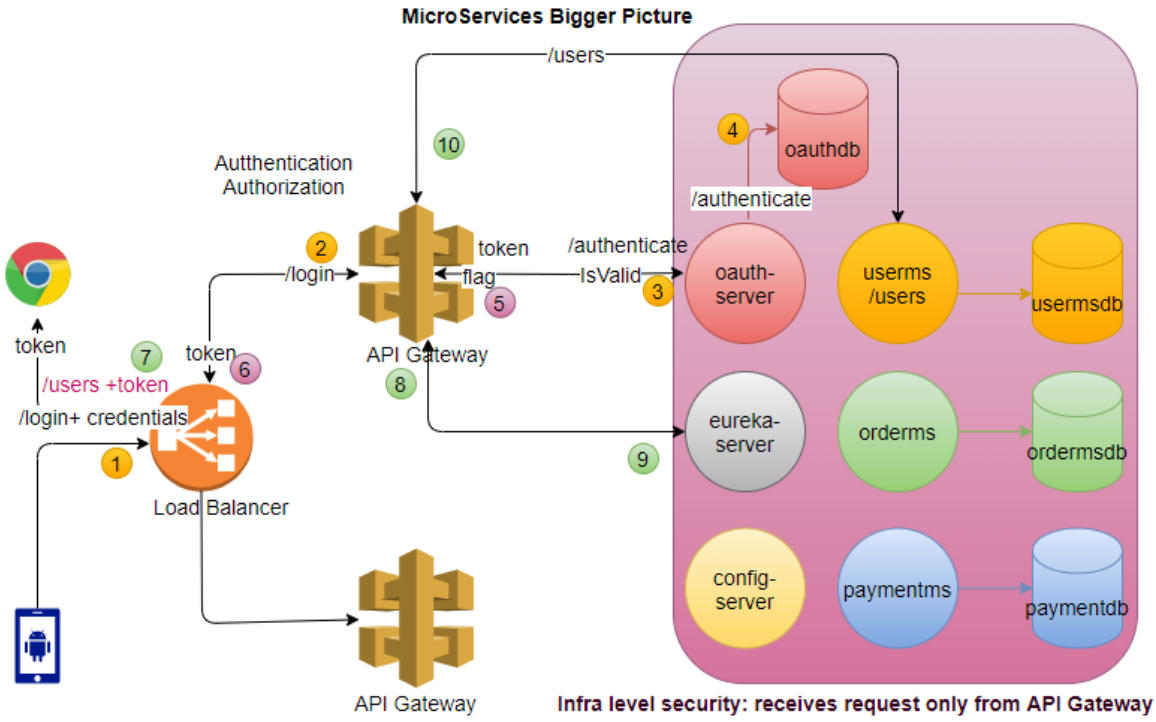
Automate policy management and configuration

Automating tasks such as defining policies, replicated across applications, separating administrative vs. development roles, and managing SSL certificates and keys.

UC: security of data



Microservices Big Picture:



step1: API Gateway receives request /login, forward the request to the oauth-server, oauth-server returns the token.
step2: User sends the request (/users) to access one of the service, API Gateway receives the request and find this request is for accessing one of the services under its control.

step3:
UC: Opaque token (Bearer):
API Gateway forward the request to oauth-server to validate the token. Oauth-server have internally
method to validate the token, if token is valid, it returns the true flag to API Gateway.
step4: If API Gateway positive response from oauth-server, it consults eureka-server about requested service (/users).
step5: eureka-server returns the url of the /userms.
step6: API Gateway calls the /userms

UC: JWT token (Bearer):

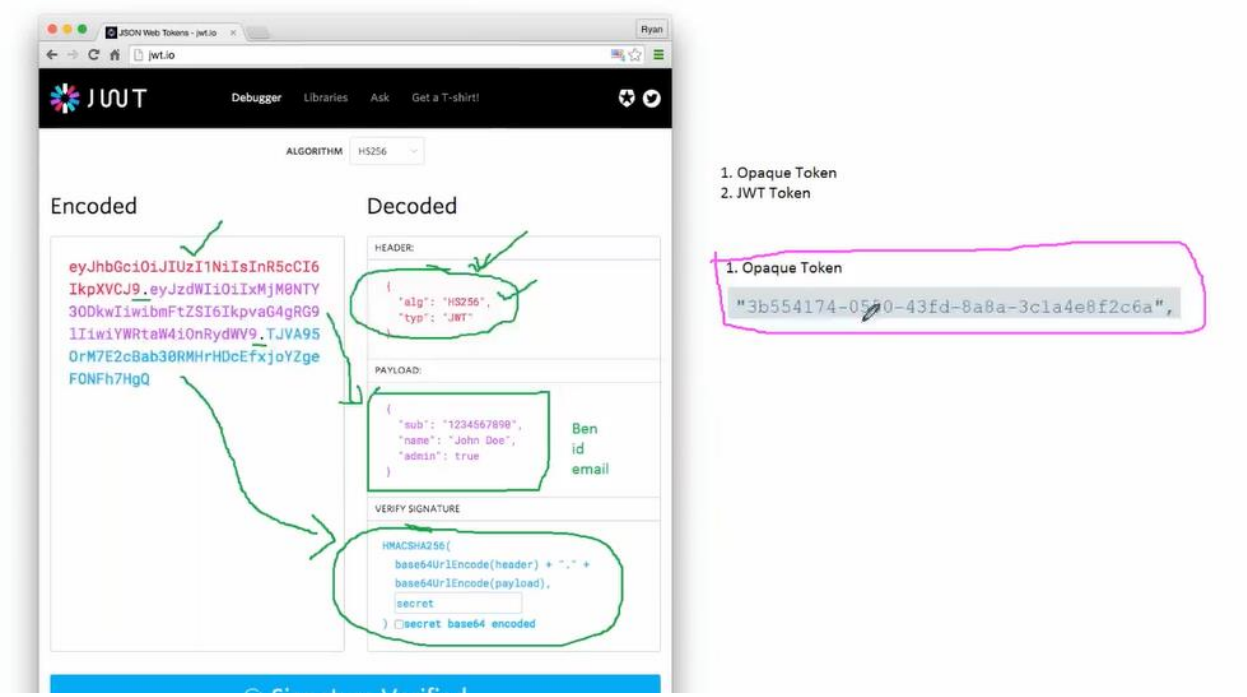
step3:

API Gateway, stores the key required for validation, validate the request itself (Improve the performance).

step4: If API Gateway positively validate the token, it consults eureka-server about requested service (/users).

step5: eureka-server returns the url of the /userms.

step6: API Gateway calls the /userms



External Load Balancer

- Apache HTTP server
- NGINX
- F5
- HA Proxy

Oauth-server implementation?

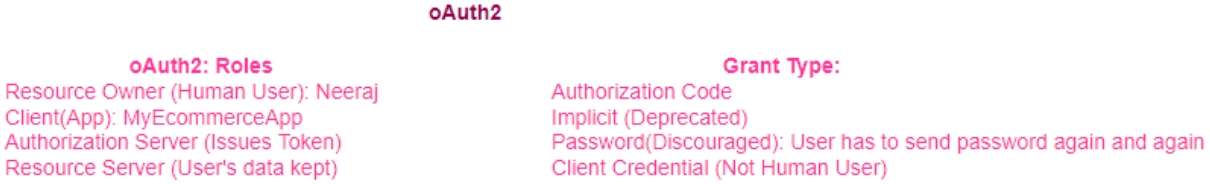
- single-signoff
- create your own oauth-server
- out of box solution to implement the security:
    - Okta
    - KeyClock

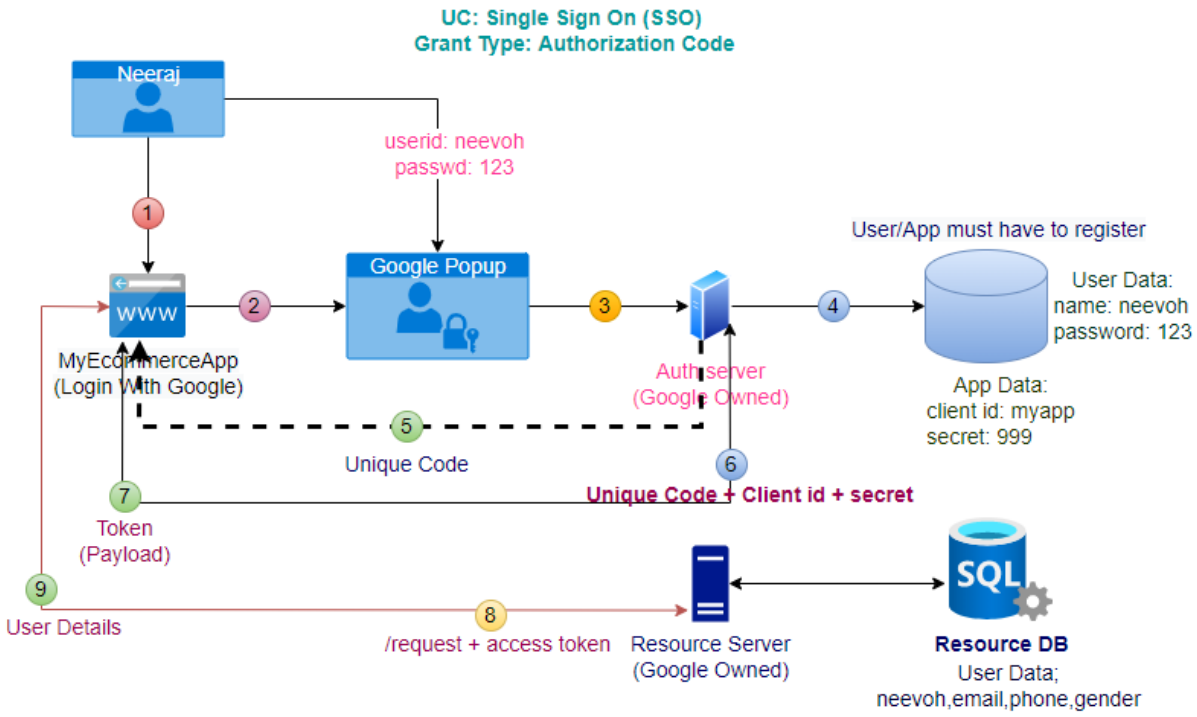
Difference between Opaque and JWT token?

Opaque Token is dump token, just a random string does not contain any inherent value.

JWT is smart token, it contains the inherent information (HEADER, PAYLOAD and VERIFY SIGNATURE).

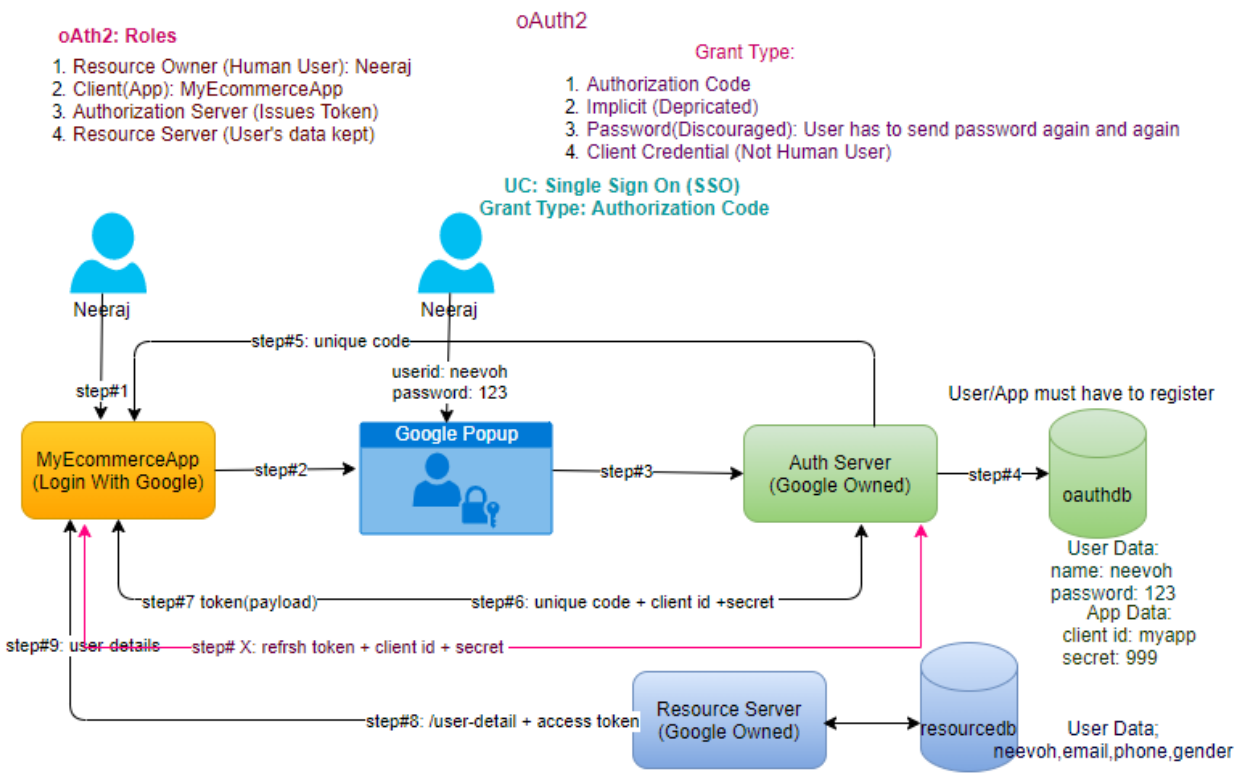
Oauth2 (Application-level Security: Third party login):





Note:

- User and Client (App) must be registered themselves.
- **Step6 and Step7 are not available in implicit grant type**,
- in implicit grant type instead of unique code, immediate token return back to the client application (less secure).
- In microservices generally uses **client credentials** grant type because its service-to-service interaction no human interaction.
- In microservices, architecture authorization server and resource server may be different.
- In token (Payload) **access writes** are defined.
- We have to make API Gateway as a resource server for external users, also can make paymentms as a resource server for internal services.
- The application sends the access token to the resource server, not the refresh token, refresh token is used by an application to request to Auth-server for a request of a new token in case of access token expires.
- Enterprise app doesn't use Google/Facebook for security, they either use their own OAuth-server or out of box solutions like **Okata, KeyClock** (manage the token).
- The second level of security we can add in microservices, userms can't call paymentms directly (need token), while orderms can directly call the paymentms without the token.
- oAuth2 is a complete flow, JWT is token



step1: User access the client application (MyEcommerceApp), have option of Login with third party (Google, GitHub, Facebook etc,)

step2: User clicks on Login with Google, Google provided Popup open.

step3: User enters Google Credentials, request goes to Auth server (Owned by Google).

step4: Authorization Server validate the credentials.

step5: Authorization Server returns the unique code back to the client application. (After this user interaction is done).

step6: Client application sends a request to Authorization Server with the request parameters (unique id + client id +secret).

step7: Authorization Server returns the token (payload) back to client application.

step8: client application sends request with token to resource server to get the user details.

step9: Resource server validate the token and after validation sends the requested details back to client application.

How Payload looks return by Auth-server?

Opaque Token:


```
{
"access_token":"3999755e-e987-4a8b-8666-109ce0261f3",
"token_type":"bearer",
"expires_in"(TTL):38,
"scope":"read write",
"custom_param": "any values",
"refresh_token":"aaaaaaa-9999999-8888888-22-22-2-2-2"
}
```


JWT:


```
{
"access_token":"
e1111111111111.3333333333.222222222222",
"token_type":"bearer",
"expires_in":38,
"scope":"read write",
"custom_param": "any values",
"refresh_token":"
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzY29wZSI6WyJyZWFkIiwid3JpdGUiXSwiZXhwIjoxNTg2NjczNjA1LCJqdGki
4Y2I3ZjA5YS1jOWI5LTQ5YzQtOTA0ZC11M2I3ZTVkMmExMmMiLCJjbGllbnRfaWQiOiJjYW5keSJ9.aCmrg9LKPkAD99v5R30I9aj
iqb4ZxWzkhjTPKacY"
}
```

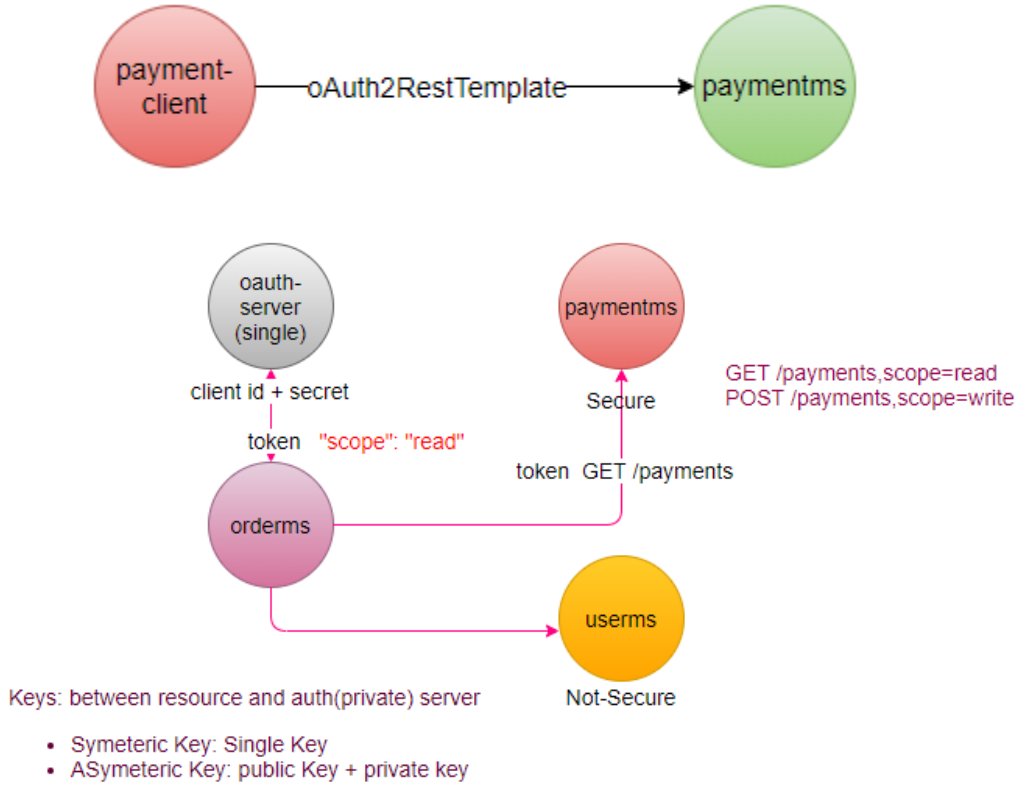

It’s end-user specific not the ms specific.

Note:

- User and Client (App) must be register themself.
- step6 and step7 are not available in implicit grant type, in implicit grant type instead of unique code, immediate token return back to the client application (less secure).
- In microservices generally uses client credentials grant type because its service-to-service interaction no human interaction.
- In microservices architecture authorization server and resource server may be different.
- In token (Payload) access writes are define.
- We have to make API Gateway as a resource server for external users, also can make paymentms as a resource server for internal services.
- application sends the access token to the resource server, not the refresh token, refresh token is used by application to request to auth-server for request of new token in case of access token expires.

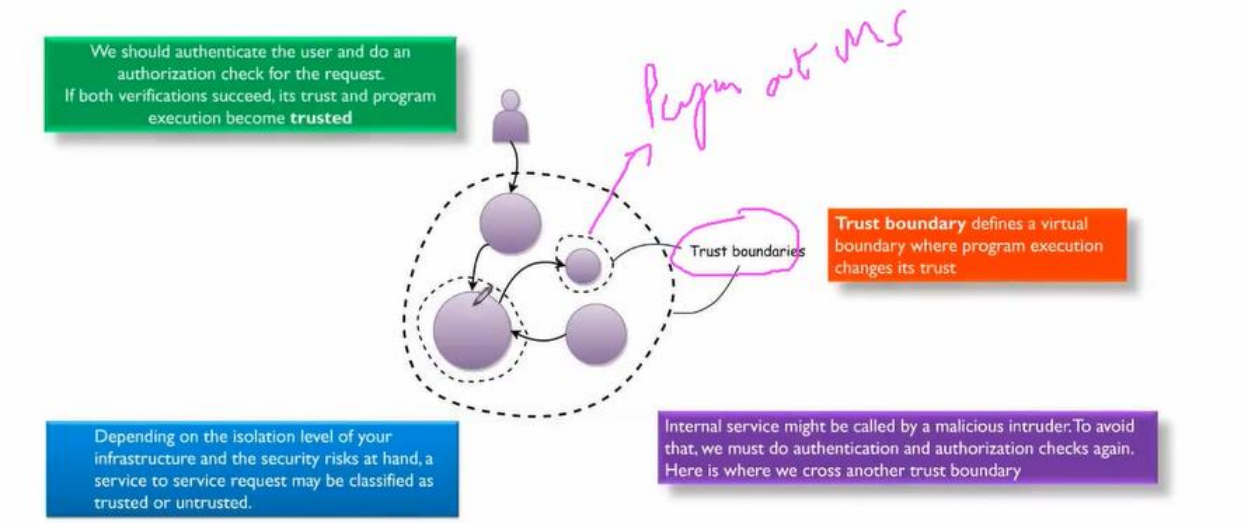
- Enterprise app doesn’t use google/Facebook for security, they either use their own oauth-server or out of box solutions like Okata, KeyClock (manage the token).
- Second level of security we can add in microservices, userms can't call paymentms directly (need token), while orderms can directly call the paymentms without the token.
- oAuth2 is a complete flow, JWT is token.

UC: security between the microservices (make some services doubly secure)



There is always two-calls (one to get the token, second to get the resource)

Trust boundaries

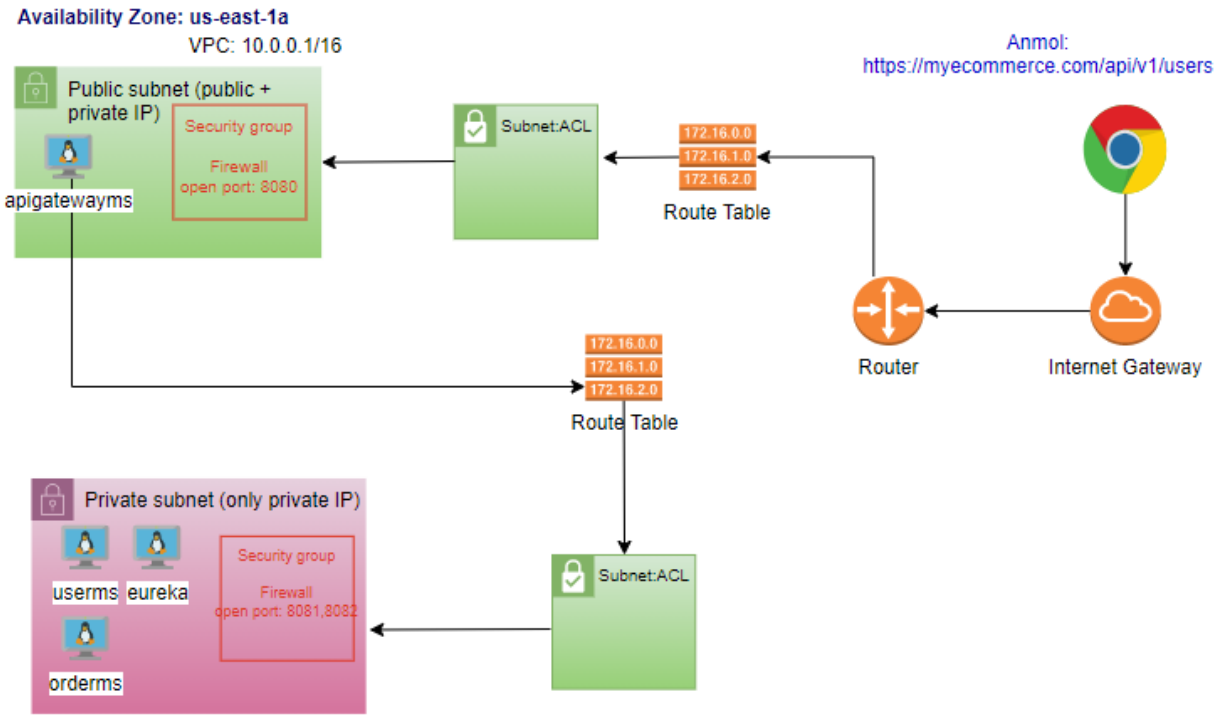


How to register client application with auth server?
GitHub-> Setting

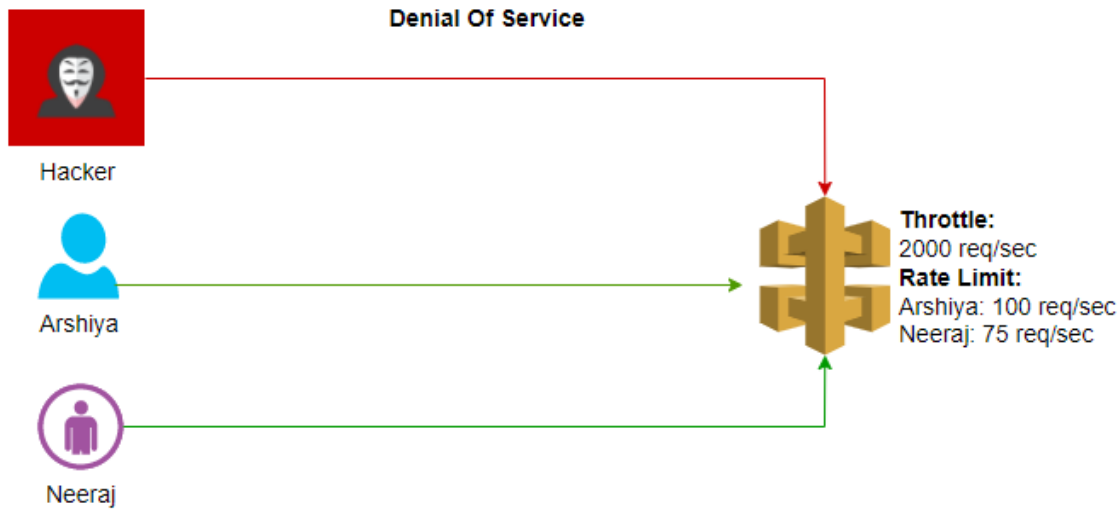
Commercial Auth Servers:

- Okta
- KeyClock

Infra Security

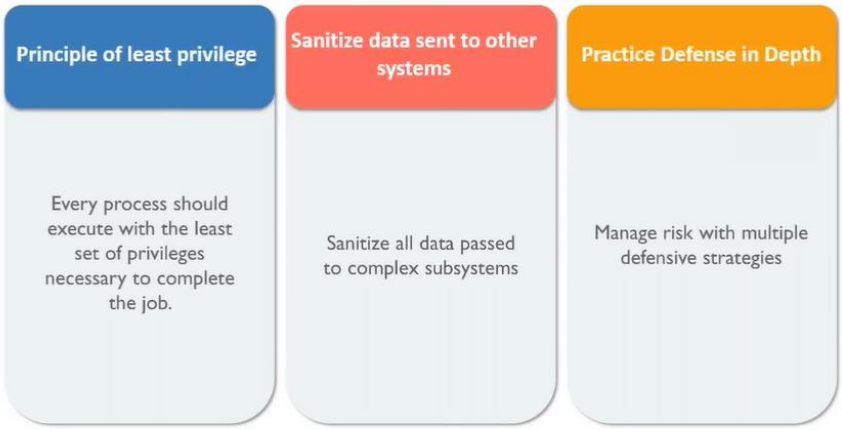


Denial of Service Attack



Use Secure Code Practice

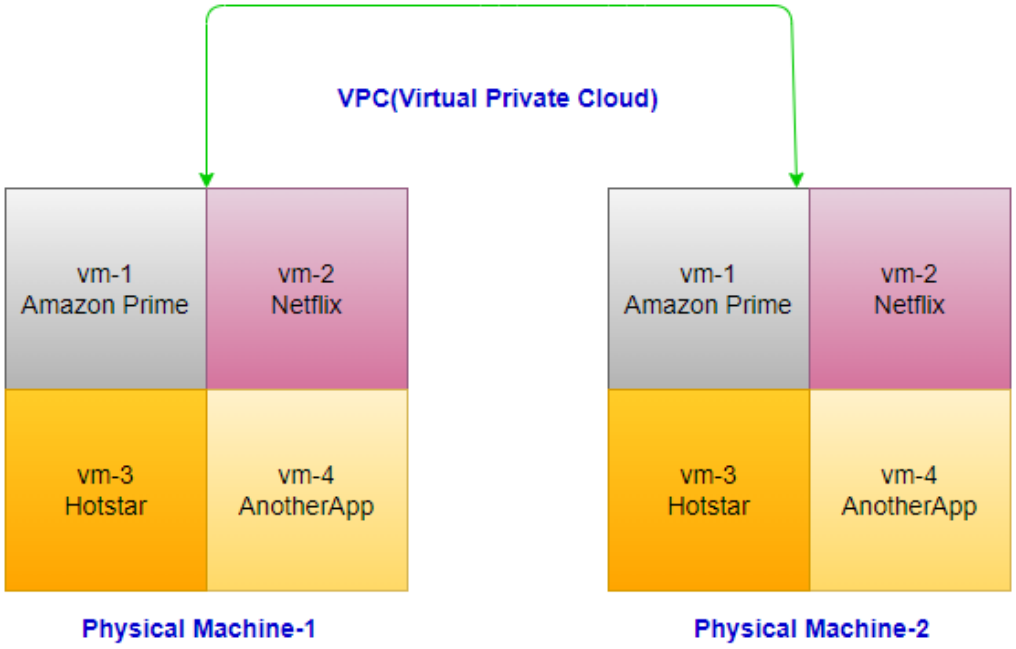
Use Secure Code Practice



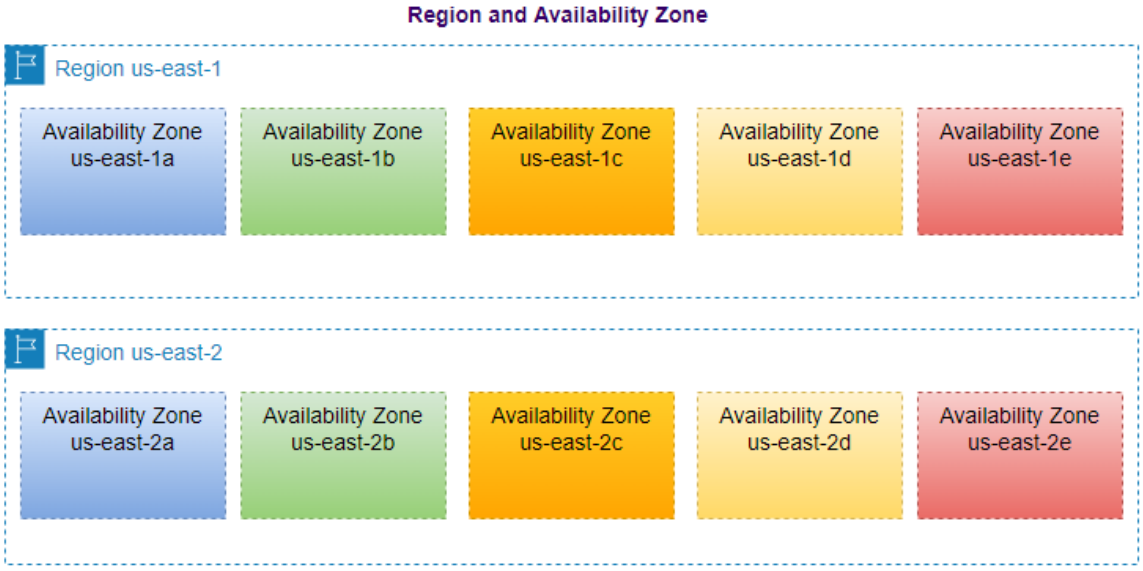

```
User.java:
- id, name, age, createdAt, createdBy, updatedAt, updatedBy,

UserTO.java -> TO = transfer object
id
```

Paymentms needs only Id
Orderms need Id and name

Cloud Intro (VPC)



Region and Availability Zone



oAuth2 can be used for SSO, JWT can be used with or without oAuth2.
Cloud provides federation for authentication/authorization.